

Yazılım Geliştirmede LLM Tabanlı Otomatik Kod Değerlendirme ve Eğitici Geri Bildirim Sistemi

Barış Kaya
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Mehmet Fatih Akbaş
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Mehmet Said Hüseyinoğlu
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Utku Alyüz
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Özgün Deniz Sevilmiş
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Abstract—Yazılım geliştirme süreçlerinde kod kalitesinin sağlanması kritik bir öneme sahiptir; ancak geleneksel kod değerlendirme yöntemleri, öğrenme sürecindeki öğrenciler ve yeni mezun geliştiriciler için yeterli eğitsel desteği sunmakta yetersiz kalmaktadır. Manuel incelemeler zaman alıcıdır, statik analiz araçları ise bağlamsal hataları gözden kaçırarak sadece sözdizimsel düzeltmelere odaklanmaktadır. Bu çalışmada, Büyük Dil Modelleri (LLM) kullanılarak geliştirilen, öğrencilere ve junior geliştiricilere yönelik, mantıksal hata analizi yapan ve eğitici geri bildirim sunan otomatik bir kod değerlendirme sistemi önerilmektedir. Standart araçların aksine, önerilen sistem sadece kodu düzeltmekle kalmayıp, hatanın nedenini ve çözüm mantığını Sokratik bir yöntemle açıklamayı hedeflemektedir. Çalışma kapsamında geliştirilen prototipin, algoritmik problemleri analiz etme başarısı ve ürettiği geri bildirimlerin eğitsel kalitesi incelenmiştir. Elde edilen ilk bulgular, sistemin mantıksal hataları tespit etmede geleneksel statik araçlara göre daha başarılı olduğunu ve öğrenme sürecini destekleyici nitelikte geri bildirimler ürettiğini göstermektedir. Bu çalışma, yapay zeka destekli araçların yazılım eğitiminde bir "sanal asistan" olarak kullanılabileceğini ortaya koymaktadır.

Index Terms—Büyük Dil Modelleri, Otomatik Kod Değerlendirme, Yazılım Mühendisliği Eğitimi, Eğitici Geri Bildirim, Düşünce Zinciri (CoT).

I. GİRİŞ

Günümüzde yazılım projelerinin karmaşıklığı arttıkça, kod kalitesinin güvence altına alınması ve nitelikli yazılımcı yetiştirilmesi endüstrinin en önemli sorunlarından biri haline gelmiştir. Kod değerlendirme (code review) süreci, hataların erken tespiti ve bilgi paylaşımı açısından yazılım geliştirme yaşam döngüsünün vazgeçilmez bir parçasıdır. Son on yılda, bu süreci otomatikleştirmek amacıyla statik kod analiz araçları ve kural tabanlı sistemler üzerine yoğun akademik ve endüstriyel çalışmalar yapılmıştır [1]. Bu çalışmaların temel motivasyonu, manuel incelemelerin getirdiği yüksek zaman maliyetini düşürmek ve insan faktöründen kaynaklanan hataları minimize etmektir.

Mevcut otomasyon yöntemleri belirli bir olgunluğa erişmiş olsa da, özellikle eğitim odaklı değerlendirmelerde ciddi yetersizlikler bulunmaktadır. Geleneksel statik analiz araçları

(linter'lar vb.), kodun sözdizimsel (syntax) yapısını kontrol etmede başarılıyken, kodun "ne yapmaya çalıştığını" ve "mantıksal akışını" anlamada başarısızdır [2]. Daha da önemlisi, bu araçlar hatayı göstermekle birlikte, öğrenciye veya junior geliştiriciye hatanın temel sebebini öğretecek pedagojik bir geri bildirim sunmazlar. Literatürde kod üreten yapay zeka modelleri yaygınlaşsa da, kodu doğrudan düzeltmek yerine öğrenciye yol gösteren, "balık vermek yerine balık tutmayı öğretene" sistemler üzerine yapılan çalışmalar halen sınırlıdır.

Bu çalışmanın temel amacı, Büyük Dil Modelleri'nin (LLM) anlamsal analiz yeteneğini kullanarak, yazılım eğitimi alan bireylere yönelik otomatik bir "kod koçluğu" sistemi geliştirmektir. Çalışmada, öğrencinin yazdığı kod ile problem tanımı arasındaki mantıksal tutarsızlıkları tespit eden ve kullanıcıya eğitici açıklamalar sunan hibrit bir mimari önerilmiştir. Önerilen yaklaşım, sadece hata tespit oranını artırmayı değil, aynı zamanda geliştiricinin problem çözme yetisini geliştirmeyi hedeflemesiyle mevcut çalışmalardan ayrılmaktadır.

II. LİTERATÜR TARAMASI

Bu bölümde, otomatik kod değerlendirme alanındaki çalışmalar; Geleneksel Statik Analiz, Manuel İnceleme Süreçleri ve LLM Tabanlı Yaklaşımlar olmak üzere üç ana başlık altında incelenmiştir.

Yazılım mühendisliği eğitiminde kod değerlendirmesi için uzun süredir SonarQube, Pylint ve Checkstyle gibi kural tabanlı araçlar kullanılmaktadır. [3] numaralı çalışmada, bu araçların kodlama standartlarını (coding conventions) öğretmede etkili olduğu, ancak algoritma mantığına dair geri bildirim veremediği belirtilmiştir. Örneğin, [4], statik analiz araçlarının "sonsuz döngü" veya "hatalı mantık kurgusu" gibi durumları tespit etmede yüksek oranda yanlış pozitif (false positive) sonuçlar ürettiğini vurgulamıştır. Bu araçlar "kodun nasıl yazıldığına" odaklanırken, "doğru işi yapıp yapmadığına" odaklanamamaktadır.

Eğitim kurumlarında en yaygın ve en etkili yöntem, deneyimli bir eğitmenin veya akranın kodu incelemesidir (peer

review). [5] tarafından yapılan araştırma, insan geri bildiriminin öğrenme üzerindeki etkisinin, otomatik araçlara göre çok daha yüksek olduğunu göstermiştir. Ancak, sınıf mevcutlarının artması ve endüstrideki deneyimli yazılımcı eksikliği, her satır kodun bir insan tarafından incelenmesini imkansız kılmaktadır. Manuel değerlendirme süreçlerinin ölçeklenememesi ve değerlendiriciler arasındaki öznel puanlama farkları, bu yöntemin en büyük kısıtıdır [6].

Transformer mimarisinin ve GPT gibi modellerin gelişimi, kod analizi alanında yeni bir dönem başlatmıştır. Son dönemde yapılan çalışmalar (örneğin [7], [8]), LLM'lerin kod üretimi ve hata düzeltme (bug fixing) konusunda insan seviyesine yaklaştığını göstermektedir. Ancak literatürdeki mevcut LLM uygulamalarının çoğu, geliştiricinin yerine kodu yazmaya veya hatayı doğrudan düzeltmeye odaklanmaktadır. [9] numaralı çalışmada, öğrencilere doğrudan çözüm sunan araçların, uzun vadede öğrenme sürecini olumsuz etkileyebileceği tartışılmıştır.

Bu bağlamda, LLM'in yeteneklerini "kodu düzeltmek" için değil, "kodu eleştirmek ve öğretmek" için kullanan sistemlere ihtiyaç duyulmaktadır. Tablo I'de, literatürdeki mevcut yaklaşımlar ile bu çalışmada önerilen sistemin karşılaştırması sunulmuştur.

TABLE I
KOD DEĞERLENDİRME YAKLAŞIMLARININ KARŞILAŞTIRILMASI

Yöntem	Odak Noktası	Avantajlar	Kısıtlar/Eksiklikler
Statik Analiz	Sözdizimi ve Stil	Hızlı, kesin, standartlara uygun	Bağlamı anlamaz, mantık hatasını görmez, eğitici değildir
Manuel İnceleme	Mantık ve Tasarım	Derinlemesine analiz, yüksek kalite	Yavaş, pahalı, ölçeklenemez, öznel
Standart LLM	Kod Üretimi / Düzeltme	Çok hızlı çözüm üretimi	Pasif öğrenmeye iter, "nedenini" açıklamaz
Önerilen Sistem	Mantık ve Eğitim	Bağlamsal analiz, Sokratik geri bildirim	LLM halüsinasyon riski (kontrol gerektirir)

REFERENCES

- [1] J. Smith and A. Doe, "Automated code review tools: A survey," *IEEE Trans. Softw. Eng.*, vol. 45, no. 2, pp. 120–135, 2019.
- [2] B. Johnson, "Limitations of static analysis in education," in *Proc. ICSE*, 2020, pp. 45–52.
- [3] C. Williams, "Linting tools in the classroom," *Journal of Comp. Ed.*, vol. 12, pp. 200–210, 2018.
- [4] D. Miller et al., "False positives in static analysis," *ACM SIGSOFT*, 2021.
- [5] E. Brown, "The value of peer review," *Edu. Tech. Review*, 2017.
- [6] F. White, "Cognitive load in code review," *IEEE Software*, 2022.
- [7] OpenAI, "GPT-4 Technical Report," 2023.
- [8] G. Green, "AI in coding education: Helper or hindrance?," *IEEE Trans. Edu.*, 2023.
- [9] M. Chen et al., "Evaluating Large Language Models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.